2.2.4 User-Server Interaction: Cookies

We mentioned above that an HTTP server is stateless.
This simplifies server design and has permitted engineers to develop high-performance Web servers that can handle thousands of simultaneous TCP connections.

However, it is often desirable for a Web site to identify users, either because the server wishes to restrict user access or because it wants to serve content as a function of the user identity.
For these purposes, HTTP uses cookies.

Cookies, defined in [RFC 6265], allow sites to keep track of users.

Most major commercial Web sites use cookies today.

As shown in Figure 2.10, cookie technology has four components:
(1) a cookie header line in the HTTP response message;
(2) a cookie header line in the HTTP request message;
(3) a cookie file kept on the user's end system and managed by the user's browser; and
(4) a back-end database at the Web site.

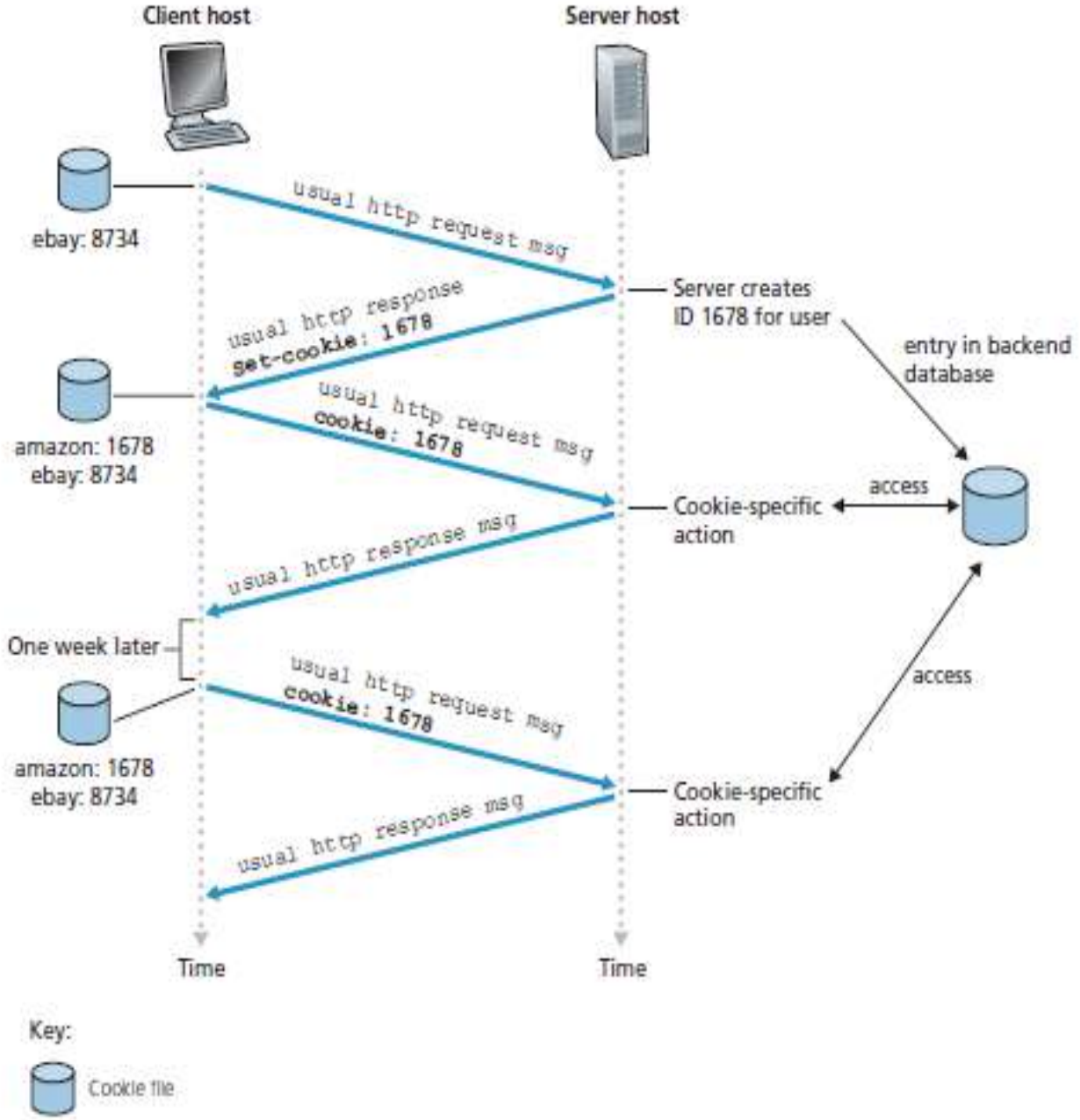


Figure 2.10 • Keeping user state with cookies

Using Figure 2.10, let's walk through an example of how cookies work.

Suppose Susan, who always accesses the Web using Internet Explorer from her home PC, contacts Amazon.com for the first time.
Let us suppose that in the past she has already visited the eBay site.

When the request comes into the Amazon Web server, the server creates a unique identification number and creates an entry in its back-end database that is indexed by the identification number. The Amazon Web server then responds to Susan's browser, including in the HTTP response a
`Set-cookie:` header, which contains the identification number.
For example, the header line might be:

```
Set-cookie: 1678
```

When Susan's browser receives the HTTP response message, it sees the `Set-cookie:` header.
The browser then appends a line to the special cookie file that it manages.
This line includes the hostname of the server and the identification number in the `Set-cookie:` header.

Note that the cookie file already has an entry for eBay, since Susan has visited that site in the past.

As Susan continues to browse the Amazon site, each time she requests a Web page, her browser consults her cookie file, extracts her identification number for this site, and puts a cookie header line that includes the identification number in the HTTP request.
Specifically, each of her HTTP requests to the Amazon server includes the header line:

Cookie: 1678

In this manner, the Amazon server is able to track Susan's activity at the Amazon site. Although the Amazon Web site does not necessarily know Susan's name, it knows exactly which pages user 1678 visited, in which order, and at what times!

Amazon uses cookies to provide its shopping cart service—Amazon can maintain a list of all of Susan's intended purchases, so that she can pay for them collectively at the end of the session.

If Susan returns to Amazon's site, say, one week later, her browser will continue to put the header line `Cookie: 1678` in the request messages.
Amazon also recommends products to Susan based on Web pages she has visited at Amazon in the past.

If Susan also registers herself with Amazon—providing full name, e-mail address, postal address, and credit card information—Amazon can then include this information in its database, thereby associating Susan's name with her identification number (and all of the pages she has visited at the site in the past!).

This is how Amazon and other e-commerce sites provide “one-click shopping”—when Susan chooses to purchase an item during a subsequent visit, she doesn’t need to re-enter her name, credit card number, or address.

From this discussion, we see that cookies can be used to identify a user.
The first time a user visits a site, the user can provide a user identification (possibly his or her name).
During the subsequent sessions, the browser passes a cookie header to the server, thereby identifying the user to the server.

Although cookies often simplify the Internet shopping experience for the user, they are controversial because they can also be considered as an invasion of privacy.
As we just saw, using a combination of cookies and user-supplied account information, a Web site can learn a lot about a user and potentially sell this information to a third party.

2.2.5 Web Caching

A **Web cache**—also called a **proxy server**—is a network entity that satisfies HTTP requests on the behalf of an origin Web server.
The Web cache has its own disk storage and keeps copies of recently requested objects in this storage.

As shown in Figure 2.11, a user's browser can be configured so that all of the user's HTTP requests are first directed to the Web cache [RFC 7234].
Once a browser is configured, each browser request for an object is first directed to the Web cache.

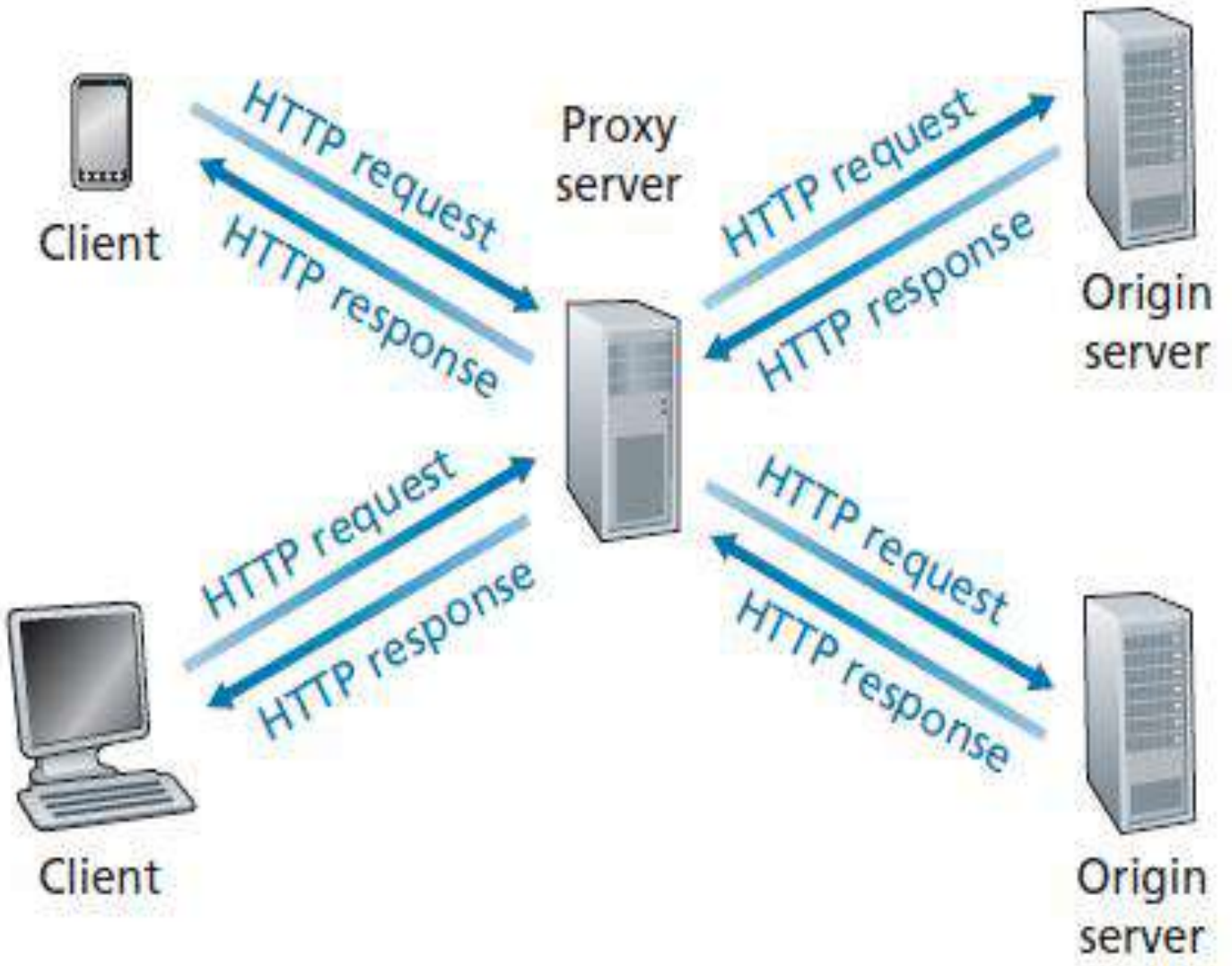


Figure 2.11 ♦ Clients requesting objects through a Web cache

As an example, suppose a browser is requesting the object `http://www.someschool.edu/campus.gif`. Here is what happens:

1. The browser establishes a TCP connection to the Web cache and sends an HTTP request for the object to the Web cache.

2. The Web cache checks to see if it has a copy of the object stored locally.
If it does, the Web cache returns the object within an HTTP response message to the client browser.

3. If the Web cache does not have the object, the Web cache opens a TCP connection to the origin server, that is, to `www.someschool.edu`.
The Web cache then sends an HTTP request for the object into the cache-to-server TCP connection.
After receiving this request, the origin server sends the object within an HTTP response to the Web cache.

4. When the Web cache receives the object, it stores a copy in its local storage and sends a copy, within an HTTP response message, to the client browser (over the existing TCP connection between the client browser and the Web cache).

Note that a cache is both a server and a client at the same time.
When it receives requests from and sends responses to a browser, it is a server.
When it sends requests to and receives responses from an origin server, it is a client.

Typically a Web cache is purchased and installed by an ISP.
For example, a university might install a cache on its campus network and configure all of the campus browsers to point to the cache.

Web caching has seen deployment in the Internet for two reasons.
First, a Web cache can substantially reduce the response time for a client request, particularly if the bottleneck bandwidth between the client and the origin server is much less than the bottleneck bandwidth between the client and the cache.
If there is a high-speed connection between the client and the cache, as there often is, and if the cache has the requested object, then the cache will be able to deliver the object rapidly to the client.

Second, as we will soon illustrate with an example, Web caches can substantially reduce traffic on an institution's access link to the Internet.
By reducing traffic, the institution (for example, a company or a university) does not have to upgrade bandwidth as quickly, thereby reducing costs.
Furthermore, Web caches can substantially reduce Web traffic in the Internet as a whole, thereby improving performance for all applications.

To gain a deeper understanding of the benefits of caches, let's consider an example in the context of Figure 2.12.
This figure shows two networks—the institutional network and the rest of the public Internet.
The institutional network is a high-speed LAN.
A router in the institutional network and a router in the Internet are connected by a 15 Mbps link.
The origin servers are attached to the Internet but are located all over the globe.

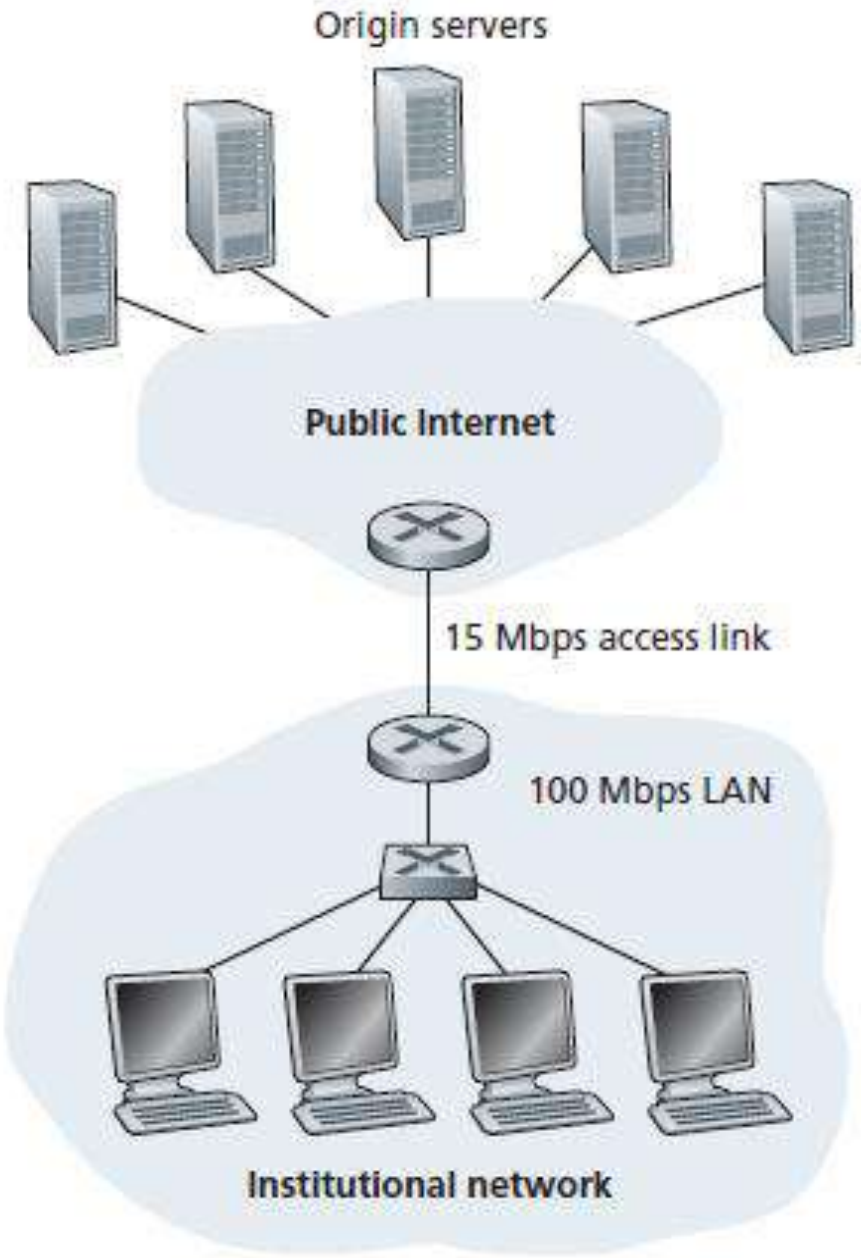


Figure 2.12 ♦ Bottleneck between an institutional network and the Internet

Suppose that the average object size is 1 Mbits and that the average request rate from the institution's browsers to the origin servers is 15 requests per second.
Suppose that the HTTP request messages are negligibly small and thus create no traffic in the networks or in the access link (from institutional router to Internet router).

Also suppose that the amount of time it takes from when the router on the Internet side of the access link in Figure 2.12 forwards an HTTP request (within an IP datagram) until it receives the response (typically within many IP datagrams) is two seconds on average.
Informally, we refer to this last delay as the "Internet delay."

The total response time—that is, the time from the browser's request of an object until its receipt of the object—is the sum of the LAN delay, the access delay (that is, the delay between the two routers), and the Internet delay.

Let's now do a very crude calculation to estimate this delay.

The traffic intensity on the LAN (see Section 1.4.2) is

$$(15 \text{ requests/sec}) \cdot (1 \text{ Mbits/request})/(100 \text{ Mbps}) = 0.15$$

whereas the traffic intensity on the access link (from the Internet router to institution router) is

$$(15 \text{ requests/sec}) \cdot (1 \text{ Mbits/request})/(15 \text{ Mbps}) = 1$$

A traffic intensity of 0.15 on a LAN typically results in, at most, tens of milliseconds of delay; hence, we can neglect the LAN delay.

However, as discussed in Section 1.4.2, as the traffic intensity approaches 1 (as is the case of the access link in Figure 2.12), the delay on a link becomes very large and grows without bound.

Thus, the average response time to satisfy requests is going to be on the order of minutes, if not more, which is unacceptable for the institution's users.

Clearly something must be done.

One possible solution is to increase the access rate from 15 Mbps to, say, 100 Mbps.
This will lower the traffic intensity on the access link to 0.15, which translates to negligible delays between the two routers.
In this case, the total response time will roughly be two seconds, that is, the Internet delay.
But this solution also means that the institution must upgrade its access link from 15 Mbps to 100 Mbps, a costly proposition.

Now consider the alternative solution of not upgrading the access link but instead installing a Web cache in the institutional network.
This solution is illustrated in Figure 2.13.

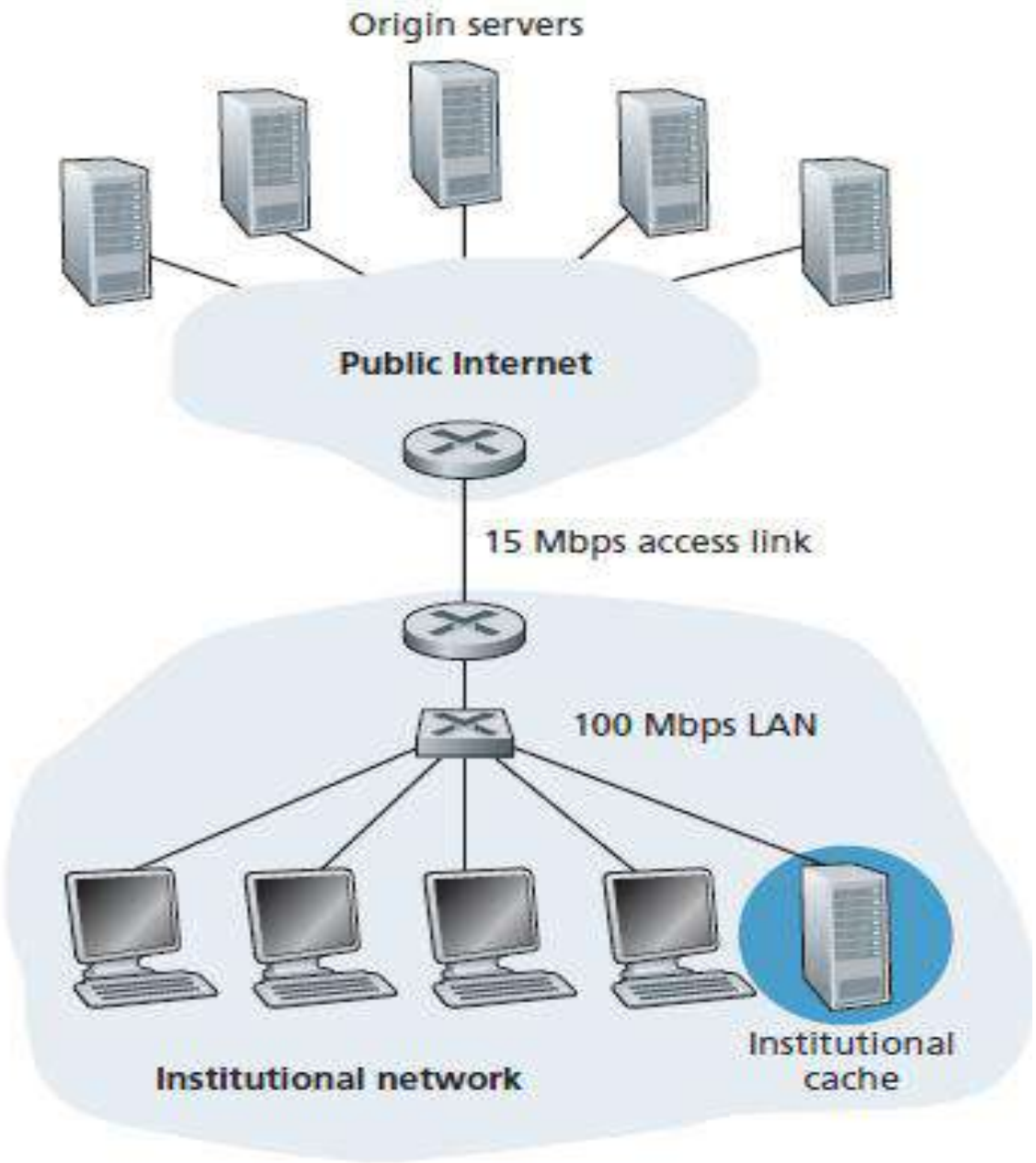


Figure 2.13 ♦ Adding a cache to the institutional network

Hit rates—the fraction of requests that are satisfied by a cache— typically range from 0.2 to 0.7 in practice.

For illustrative purposes, let's suppose that the cache provides a hit rate of 0.4 for this institution. Because the clients and the cache are connected to the same high-speed LAN, 40 percent of the requests will be satisfied almost immediately, say, within 10 milliseconds, by the cache.

Nevertheless, the remaining 60 percent of the requests still need to be satisfied by the origin servers. But with only 60 percent of the requested objects passing through the access link, the traffic intensity on the access link is reduced from 1.0 to 0.6.
Typically, a traffic intensity less than 0.8 corresponds to a small delay, say, tens of milliseconds, on a 15 Mbps link.
This delay is negligible compared with the two-second Internet delay.
Given these considerations, average delay therefore is

$$0.4 \cdot (0.01 \text{ seconds}) + 0.6 \cdot (2.01 \text{ seconds})$$

which is just slightly greater than 1.2 seconds.

Thus, this second solution provides an even lower response time than the first solution, and it doesn't require the institution to upgrade its link to the Internet.
The institution does, of course, have to purchase and install a Web cache.
But this cost is low

Through the use of **Content Distribution Networks (CDNs)**, Web caches are increasingly playing an important role in the Internet.
A CDN company installs many geographically distributed caches throughout the Internet, thereby localizing much of the traffic.

We will discuss CDNs in more detail in Section 2.6.

The Conditional GET

Although caching can reduce user-perceived response times, it introduces a new problem—the copy of an object residing in the cache may be stale.
In other words, the object housed in the Web server may have been modified since the copy was cached at the client.

Fortunately, HTTP has a mechanism that allows a cache to verify that its objects are up to date.
This mechanism is called the **conditional GET** [RFC 7232].

An HTTP request message is a so-called conditional GET message if
(1) the request message uses the `GET` method and (2) the request message includes an `If-Modified-Since:` header line.

To illustrate how the conditional GET operates, let's walk through an example.
First, on the behalf of a requesting browser, a proxy cache sends a request message to a Web server:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

Second, the Web server sends a response message with the requested object to the cache:

```
HTTP/1.1 200 OK
Date: Sat, 3 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 9 Sep 2015 09:23:24
Content-Type: image/gif
(data data data data data ...)
```

The cache forwards the object to the requesting browser but also caches the object locally. Importantly, the cache also stores the last-modified date along with the object.

Third, one week later, another browser requests the same object via the cache, and the object is still in the cache.
Since this object may have been modified at the Web server in the past week, the cache performs an up-to-date check by issuing a conditional GET.
Specifically, the cache sends:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015
09:23:24
```

Note that the value of the `If-modified-since:` header line is exactly equal to the value of the `Last-Modified:` header line that was sent by the server one week ago. This conditional GET is telling the server to send the object only if the object has been modified since the specified date.

Suppose the object has not been modified since 9 Sep 2015 09:23:24.
Then, fourth, the Web server sends a response message to the cache:

```
HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)!
(empty entity body)
```

We see that in response to the conditional GET, the Web server still sends a response message but does not include the requested object in the response message.
Including the requested object would only waste bandwidth and increase user-perceived response time, particularly if the object is large.
Note that this last response message has `304 Not Modified` in the status line, which tells the cache that it can go ahead and forward its (the proxy cache's) cached copy of the object to the requesting browser.

2.2.6 HTTP/2

HTTP/2 [RFC 7540], standardized in 2015, was the first new version of HTTP since HTTP/1.1, which was standardized in 1997.
Since standardization, HTTP/2 has taken off, with over 40% of the top 10 million websites supporting HTTP/2 in 2020 [W3Techs].
Most browsers—including Google Chrome, Internet Explorer, Safari, Opera, and Firefox—also support HTTP/2.

The primary goals for HTTP/2 are to reduce perceived latency by enabling request and response multiplexing over a *single* TCP connection, provide request prioritization and server push, and provide efficient compression of HTTP header fields.

HTTP/2 does not change HTTP methods, status codes, URLs, or header fields.
Instead, HTTP/2 changes how the data is formatted and transported between the client and server.

To motivate the need for HTTP/2, recall that HTTP/1.1 uses persistent TCP connections, allowing a Web page to be sent from server to client over a single TCP connection.
By having only one TCP connection per Web page, the number of sockets at the server is reduced and each transported Web page gets a fair share of the network bandwidth (as discussed below).

But developers of Web browsers quickly discovered that sending all the objects in a Web page over a single TCP connection has a **Head of Line (HOL) blocking** problem.
To understand HOL blocking, consider a Web page that includes an HTML base page, a large video clip near the top of Web page, and many small objects below the video.
Further suppose there is a low-to-medium speed bottleneck link (for example, a low-speed wireless link) on the path between server and client.
Using a single TCP connection, the video clip will take a long time to pass through the bottleneck link, while the small objects are delayed as they wait behind the video clip; that is, the video clip at the head of the line blocks the small objects behind it.

HTTP/1.1 browsers typically work around this problem by opening multiple parallel TCP connections, thereby having objects in the same web page sent in parallel to the browser. This way, the small objects can arrive at and be rendered in the browser much faster, thereby reducing user-perceived delay.
TCP congestion control, discussed in detail in Chapter 3, also provides browsers an unintended incentive to use multiple parallel TCP connections rather than a single persistent connection.

Very roughly speaking, TCP congestion control aims to give each TCP connection sharing a bottleneck link an equal share of the available bandwidth of that link; so if there are n TCP connections operating over a bottleneck link, then each connection approximately gets $1/n$th of the bandwidth.

By opening multiple parallel TCP connections to transport a single Web page, the browser can "cheat" and grab a larger portion of the link bandwidth.
Many HTTP/1.1 browsers open up to six parallel TCP connections not only to circumvent HOL blocking but also to obtain more bandwidth.

One of the primary goals of HTTP/2 is to get rid of (or at least reduce the number of) parallel TCP connections for transporting a single Web page.
This not only reduces the number of sockets that need to be open and maintained at servers, but also allows TCP congestion control to operate as intended.
But with only one TCP connection to transport a Web page, HTTP/2 requires carefully designed mechanisms to avoid HOL blocking.

HTTP/2 Framing

The HTTP/2 solution for HOL blocking is to break each message into small frames, and interleave the request and response messages on the same TCP connection.

To understand this, consider again the example of a Web page consisting of one large video clip and, say, 8 smaller objects.

Thus the server will receive 9 concurrent requests from any browser wanting to see this Web page.

For each of these requests, the server needs to send 9 competing HTTP response messages to the browser.

Suppose all frames are of fixed length, the video clip consists of 1000 frames, and each of the smaller objects consists of two frames.

With frame interleaving, after sending one frame from the video clip, the first frames of each of the small objects are sent.

Then after sending the second frame of the video clip, the last frames of each of the small objects are sent.

Thus, all of the smaller objects are sent after sending a total of 18 frames.

If interleaving were not used, the smaller objects would be sent only after sending 1016 frames.

Thus the HTTP/2 framing mechanism can significantly decrease user-perceived delay.

HTTP/2 Framing

The HTTP/2 solution for HOL blocking is to break each message into small frames, and interleave the request and response messages on the same TCP connection.

To understand this, consider again the example of a Web page consisting of one large video clip and, say, 8 smaller objects.

Thus the server will receive 9 concurrent requests from any browser wanting to see this Web page.

For each of these requests, the server needs to send 9 competing HTTP response messages to the browser.

Suppose all frames are of fixed length, the video clip consists of 1000 frames, and each of the smaller objects consists of two frames.

With frame interleaving, after sending one frame from the video clip, the first frames of each of the small objects are sent.

Then after sending the second frame of the video clip, the last frames of each of the small objects are sent.

Thus, all of the smaller objects are sent after sending a total of 18 frames.

If interleaving were not used, the smaller objects would be sent only after sending 1016 frames.

Thus the HTTP/2 framing mechanism can significantly decrease user-perceived delay.

The ability to break down an HTTP message into independent frames, interleave them, and then reassemble them on the other end is the single most important enhancement of HTTP/2.

Similarly, a client's HTTP requests are broken into frames and interleaved.

In addition to breaking down each HTTP message into independent frames, the framing sublayer also binary encodes the frames.
Binary protocols are more efficient to parse, lead to slightly smaller frames, and are less error-prone.

Response Message Prioritization and Server Pushing

Message prioritization allows developers to customize the relative priority of requests to better optimize application performance.

As we just learned, the framing sub-layer organizes messages into parallel streams of data destined to the same requestor.
When a client sends concurrent requests to a server, it can prioritize the responses it is requesting by assigning a weight between 1 and 256 to each message.
The higher number indicates higher priority.
Using these weights, the server can send first the frames for the responses with the highest priority.

Another feature of HTTP/2 is the ability for a server to send multiple responses for a single client request.
That is, in addition to the response to the original request, the server can *push* additional objects to the client, without the client having to request each one.
This is possible since the HTML base page indicates the objects that will be needed to fully render the Web page.
So instead of waiting for the HTTP requests for these objects, the server can analyze the HTML page, identify the objects that are needed, and send them to the client *before receiving explicit requests for these objects*.
Server push eliminates the extra latency due to waiting for the requests.

HTTP/3

QUIC, discussed in Chapter 3, is a new "transport" protocol that is implemented in the application layer over the bare-bones UDP protocol.
QUIC has several features that are desirable for HTTP, such as message multiplexing (interleaving), per-stream flow control, and low-latency connection establishment.

HTTP/3 is yet a new HTTP protocol that is designed to operate over QUIC.
As of 2020, HTTP/3 is described in Internet drafts and has not yet been fully standardized.

Many of the HTTP/2 features (such as message interleaving) are subsumed by QUIC, allowing for a simpler, streamlined design for HTTP/3.